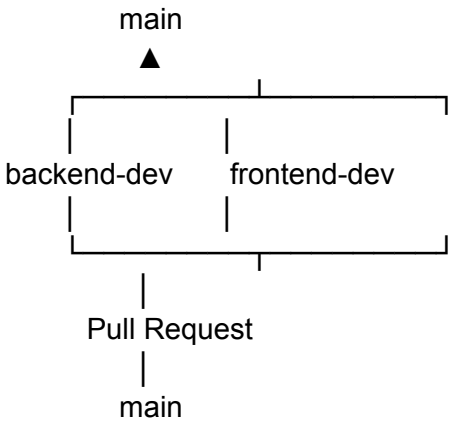
🌿 Git Workflow



Backend Developer Workflow (Linux)

Every Morning (Before Starting Work)

Step 1

Go to project

cd ~/Documents/LavernaEvents_Project/lavernaevents

Step 2

Go to main

git checkout main

Step 3

Download latest code

git pull origin main

Step 4

Switch back

git checkout backend-dev

Step 5

Update backend-dev

git merge main

If there are no conflicts

Already up to date.

Step 6

Push updated branch (optional but recommended)

git push origin backend-dev

Step 7

Run Backend

cd backend

source venv/bin/activate

python manage.py migrate

python manage.py runserver

Step 8

Run Frontend

Open another terminal

cd frontend

npm install

npm run dev

Now start development.

Backend Developer (After Finishing Work)

Go to project

cd ~/Documents/LavernaEvents_Project/lavernaevents

Check changes

git status

Stage

git add .

Commit

git commit -m "Implement user registration API"

Push

git push origin backend-dev

Go to GitHub

Create Pull Request

backend-dev

↓

main

Merge Pull Request.

Frontend Developer Workflow (Windows)

Every Morning

Open PowerShell

Go to project

cd Documents\LavernaEvents_Project\lavernaevents

Go to main

git checkout main

Download latest

git pull origin main

Return

git checkout frontend-dev

Merge latest main

git merge main

Push updated branch (optional)

git push origin frontend-dev

Run Backend

cd backend

.\venv\Scripts\Activate

python manage.py migrate

python manage.py runserver

Open another PowerShell

Run Frontend

cd frontend

npm install

npm run dev

Start development.

Frontend Developer (After Finishing Work)

Go to project

cd Documents\LavernaEvents_Project\lavernaevents

Check

git status

Stage

git add .

Commit

git commit -m "Build registration page"

Push

git push origin frontend-dev

Go to GitHub

Create Pull Request

frontend-dev

↓

main

Merge.

When One Developer Merges to `main`

Suppose the **Backend Developer** merged new code into `main`.

The **Frontend Developer** should **not** pull `backend-dev`.

Instead, they should update from `main`:

git checkout main

git pull origin main

git checkout frontend-dev

git merge main

Likewise, if the **Frontend Developer** merges to `main`, the **Backend Developer** updates from `main`:

git checkout main

git pull origin main

git checkout backend-dev

git merge main

Git Commands Cheat Sheet

Before Starting Work

Backend (Linux)

git checkout main

git pull origin main

git checkout backend-dev

git merge main

git push origin backend-dev

Frontend (Windows)

git checkout main

git pull origin main

git checkout frontend-dev

git merge main

git push origin frontend-dev

After Finishing Work

Backend

git status

git add .

git commit -m "Meaningful commit message"

git push origin backend-dev

Frontend

git status

git add .

git commit -m "Meaningful commit message"

git push origin frontend-dev

Then:

- Open GitHub
- Create a Pull Request to `main`
- Review (if needed)
- Merge
- Delete only temporary feature branches (if you use them)

Rules

✅ Always start from the latest `main`.

✅ Always work on your own development branch.

✅ Never commit directly to `main`.

✅ Merge into `main` only through Pull Requests.

✅ After someone merges to `main`, the other developer updates **from `main`**, not from the other developer's branch.

This workflow keeps both developers synchronized and minimizes merge conflicts while maintaining a stable `main` branch.